



Designing Agent-Oriented Systems

Massimo Cossentino¹ , Luca Sabatucci¹ , and Valeria Seidita²

¹ ICAR, National Research Council (CNR), Palermo, Italy
{massimo.cossentino,luca.sabatucci}@icar.cnr.it

² Department of Engineering, University of Palermo, Palermo, Italy
valeria.seidita@unipa.it

Abstract. In the era of computational complexity, autonomous agent-based systems stand as an established paradigm that simplifies various stages of project conception and design compared to classical approaches. This chapter explores the rich trajectory of Agent-Oriented Software Engineering (AOSE), delineating its path from conceptual foundations to contemporary applications with a specific focus on the contributions presented during Workshop on Objects and Agents (WOA). Our study begins with a retrospective analysis of how AOSE methodologies have evolved to implement increasingly advanced agents. We trace this progression from methodologies designed for simple state machine-based reactive systems through those supporting cognitive agents to sophisticated approaches enabling the development of Belief-Desire-Intention (BDI) models. This evolution culminates in contemporary methodologies that facilitate the creation of complex agents capable of real-time introspection, reflection, and self-adaptation.

The second part of this chapter analyzes the core principles and practices of AOSE methodologies and their development and how they evolved during WOA events. When creating an agent methodology, we emphasise that a good principle is to follow clearly defined rules that consider appropriate abstractions for each context, making it possible to move from requirements to code when applying the methodology. Also, we underline and explore that essential tools for this are the metamodel and a notation language suitable for developing models for each phase of the methodology, as well as the selected framework or implementation platform to be used.

We conclude with a forward-looking perspective on AOSE, outlining innovative directions and potential research areas.

Keywords: Agent-Oriented Software Engineering · Agent Design · Methodologies

1 Introduction

The proliferation of complex autonomous agent-based systems has become increasingly important in addressing problems characterized by intricate interactions, dynamic environments, and decentralized resource distribution, which

cannot be effectively tackled using classical methodologies. Over the years, Multi-Agent Systems (MASs) have been successfully applied across various domains, including: Smart Cities [9,30], where MASs are used to optimize energy consumption, traffic management, waste disposal, and public services, thereby contributing to sustainable and efficient city operations. Disaster Response [38,51], in which MASs play a vital role in disaster management by coordinating emergency services, allocating resources, and optimizing response strategies. Robotics, where MASs enable collaborative tasks, such as search and rescue missions, by facilitating coordination among multiple robots to achieve complex objectives [10,52]. Within Transportation systems [81], MASs manage traffic flow, route optimization, and vehicle coordination, enhancing efficiency and safety in dynamic environments. In economic contexts [16,60], MASs model complex market interactions, resource allocation, and economic phenomena. These systems help analyze market dynamics and optimize decision-making processes. Researchers also apply MASs in social structure modelling [80,87], studying collective behaviour and analyzing social networks. These systems help the designer understand emergent patterns and interactions.

Throughout all these years, it has become increasingly evident that design methodologies for agent-based systems are crucial for developing Multi-Agent Systems in such complex and mission-critical contexts. This claim is supported by the definition of many different agent-oriented methodologies over the years, answering the need for concrete support when working with high-level abstractions. As a result, there has been an evolution in modelling techniques from those treating agents as state machines to those focusing on social interactions and rational decision-making. This development has also given rise to analyzing the principles of the design process and its constituent components. The method engineering research area [22], understanding recurring patterns in methodology's rules, metamodel, and notation, has enabled the organization of a repository of fragments and provided composition guidelines for building ad-hoc methodologies.

This chapter provides an overview of agent-oriented design methodologies. The first part (Sect. 3) explores how the evolution of these methodologies followed the advancements in agent-oriented programming frameworks, from state charts to intentional systems. The section discusses design principles by emphasizing features such as autonomy, proactiveness, situatedness, and sociality. The second part of this chapter (Sect. 4) examines the fundamental principles that underlie agent-oriented methodologies, including their theoretical foundations, metamodels, guidelines and notations. Finally, this chapter concludes with some insights into future challenges and opportunities in the agent-oriented software engineering (AOSE) research area (Sect. 5).

2 AOSE: The WOA Perspective

This chapter's statistical trends and patterns have been extracted from WOA publications. However, their significance extends far beyond merely surveying

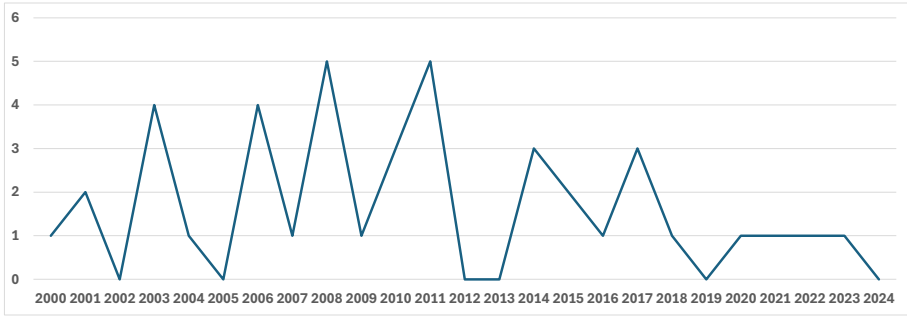


Fig. 1. Number of Agent-Oriented Software Engineering related papers at WOA per year

Italian research activities. The chapter presents a comprehensive and up-to-date perspective on international research in agent-oriented software engineering, incorporating and analyzing numerous non-Italian works that constitute the broader context of this survey. The alignment between WOA trends and international developments is particularly noteworthy, as it demonstrates not only the integration of Italian research within the global scientific community but often its anticipatory nature in relation to international trends and visions. We want to emphasize how the trends observed in WOA mirror (in many cases anticipate) international developments.

Figure 1 shows a year-by-year evolution of AOSE Publications in 25 years. The analysis of AOSE publications reveals notable fluctuations in research productivity (with peaks in 2008 and 2011). These spikes likely reflect significant developments in the field. The alternating high and low publication periods suggest a dynamic research environment that cycles between theoretical advancement and practical implementation. A recent trend of decreasing publications, especially from 2018, may indicate the field’s maturation and shifting focus from foundational theory to application and refinement.

Figure 2 reveals significant patterns in research intensity over five-year periods. The data demonstrates a notable concentration of research activity during the decade spanning 2005–2014, which accounts for approximately half of all publications. This period could represent the apex of AOSE research activity at WOA, suggesting a mature phase in the field’s development where key methodological approaches were being actively developed and refined. The subsequent gradual decline in publication volume, particularly evident in the 2020–2024 period, which represents only 10% of total publications, may indicate a shift in research focus or the consolidation of established methodologies rather than necessarily suggesting diminished importance of the field.

Figure 3 proposes a thematic distribution of AOSE Research. The categorization of AOSE papers provides valuable insights into the field’s primary research concerns and theoretical foundations. The predominance of Autonomy and Proactivity research, comprising 24% publications, reflects the field’s funda-

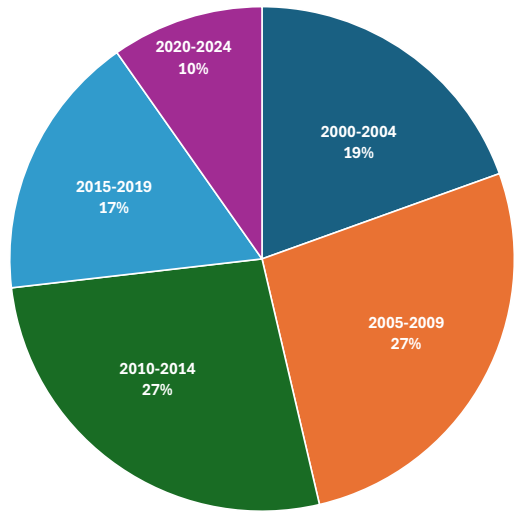


Fig. 2. Number of Agent-Oriented Software Engineering papers at WOA per lustrum

mental focus on developing systems capable of independent decision-making and goal-directed behaviour. This emphasis is complemented by substantial attention to Sociality (21%), underscoring the importance of inter-agent interactions and collaborative behaviours in Multi-Agent Systems. The significant representation

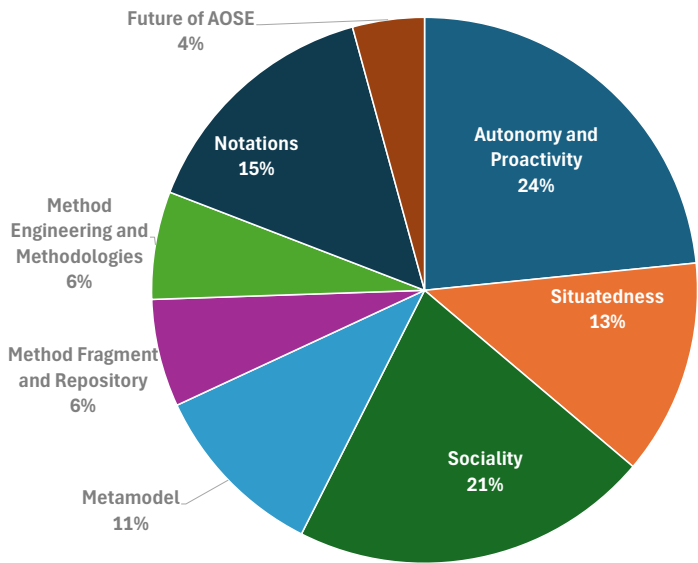


Fig. 3. Agent-Oriented Software Engineering papers at WOA by category

Table 1. WOA AOSE related papers organized according to year of publication and research category (papers may belong to more than one category).

	2000–2004	2005–2009	2010–2014	2015–2019	2020–2024	Total
Autonomy and Proactivity	[34, 78]	[45, 57, 65, 70, 84]	[82]	[14, 36, 41]		11
Situatedness	[73]	[93]	[66, 71, 82, 95]			6
Sociality	[34, 40, 73]	[44, 93]	[82, 95]	[30, 80]	[37]	10
Metamodel	[28, 61, 78]	[45]			[16]	5
Method Fragment and Repository		[68, 89]	[69]			3
Method Engineering and Methodologies	[46, 56]	[25]				3
Notation	[12, 40]		[35, 39, 82]	[29]	[38]	7
Future of AOSE				[94]	[90]	2

of Notations (15%) in the research corpus highlights the ongoing effort to develop and refine formal representations for agent-based systems. The equal distribution between Method Engineering and Method Fragment and Repository (6% each) indicates a balanced approach to methodology development, combining theoretical frameworks with practical implementation concerns. The presence of research on Situatedness (13%) demonstrates recognition of the importance of context-awareness in agent systems.

Table 1 details the distribution of AOSE papers over five-year periods. The data confirms the field’s peak activity during 2005–2014, with each lustrum accounting for 25% of total publications. This refined analysis suggests a more gradual decline in research output than initially indicated, pointing to a measured evolution rather than a sharp decrease in scholarly attention. The sustained level of publications during these peak periods suggests a robust research community actively engaging with fundamental questions in agent-oriented software engineering.

The remaining part of this chapter examines how local developments in agent-oriented software engineering in Italy relate to global research advances. It highlights the contributions of Italian researchers and the bidirectional exchange of ideas between local and international venues.

3 Designing Agents: From State-Charts to Intentional Systems

From the beginning of Multi-Agent Systems, the objective has been to devise a decentralised computational model capable of acting proactively and adapting autonomously to changing circumstances. During the mid-1990s, Jennings and Wooldridge envisioned the grounding principles that became a research agenda of

the (multi-)agent systems community [63]. Among these principles, some (autonomy, proactivity, situations, and sociality) directly involve the ability to dynamically alter agents' behaviour in response to changes in conditions, uncertainties, and objectives.

This section explores agent-oriented methodologies, aiming to show how the evolution of this research area progresses through significant milestones closely related to the characteristics of agent-based (or agent-oriented) systems.

This analysis focuses on the perspectives relevant to implementing properties like autonomy, proactivity, and situatedness. The following subsections will explore how well-known agent-oriented design methodologies propose dealing with properties.

3.1 Autonomy and Proactivity

Autonomy and proactivity are crucial features of Multi-Agent Systems. Initially, AOSE methods [59] focused more on autonomy, allowing the design of complex agent structures and internal functionalities that adhere to the agent's perceive-decide-act (PDA) cycle [83]. At the end of the 90s, the most diffused frameworks for agent development (such as FIPA-OS and JADE) conceived them as classes. The common approach to designing an autonomous behaviour was to model agents as state charts, with GAIA [92], INGENIAS [77], and PASSI [43] falling into this category.

The GAIA methodology [92] was introduced in 1999 to develop a generic architecture for information availability. This architecture aims to make it easier to locate, deliver, and manage information, content, and digital services. GAIA's functional architecture is built on Functional Units that act as building blocks for a scalable brokerage model. A few years later, two other methodologies were presented for modelling generic Multi-Agent Systems: INGENIAS [77] and PASSI [43]. Both of them adopted Unified Modeling Language (UML) [18] and Model-Driven Engineering (MDE) [88] within their process to design a Multi-Agent System as a complex software system that will exhibit high-level properties (like autonomy and proactivity).

In these years, FIPA [34,72] (Foundation for Intelligent Physical Agents) was pushing the definition of standards for developing distributed MAS. Many FIPA-compliant development platforms, including JADE, largely supported a state-based agent model. Many contributions have been presented at WOA on this topic, and this research stream arrived at a formalization of JADE presented in [14]. Such formalization includes the identification of five main entities: Multi-Agent System, agents and their classes, behaviours and their classes. The paper also provides an example of the transition system that can be obtained from a very simple JADE agent.

The great advantage of adopting a deterministic state-chart model for modelling the agent is to inherit decades of experiences from object orientation. The use of UML (as in INGENIAS and PASSI) is an example. Moreover, using model-driven engineering facilitates the development of supporting IDEs and tools. An interesting tool (ELDATool [57]) has been presented in WOA 2007.

It has been conceived for fast prototyping of agents starting from GAIA/PASSI state-charts models (for the ELDA agent framework). Another example is [84], where a repository of design patterns is presented to solve typical problems of agent design and development.

The following step in AOSE research focused on proactive agents, i.e. agents requiring a clear understanding of their own design objectives. To this purpose, goals became first-class abstractions of agent-oriented design. The TROPOS [21] methodology introduced this concept, allowing designers to reason about intentions rather than tasks, especially when analysing user requirements. In WOA 2001, TROPOS was presented as a methodology for modelling socio-technical systems, highlighting how humans and agents are integral parts of the system along with hardware and software components [45, 78].

In the same direction, at WOA 2006, the PRACTIONIST framework was presented [70]. It recognises that goals are useful and stable abstractions in the analysis and design of software applications. According to the Belief-Desire-Intention (BDI) model, it supports a goal-oriented approach to developing agent systems. At WOA 2008, [65] introduces the PRACTIONIST Studio, an integrated design and development tool tailored explicitly for designing BDI agent systems. This environment provides tools and facilities to represent the fundamental concepts and intentional elements underlying the BDI models.

Over the years, the community has realised that the agent's conception as a class needs to be revised (at that time, most of the BDI frameworks were built on top of JADE). In 2005, [19] introduced Jason, an implementation of the AgentSpeak language, an elegant agent-oriented programming language based on logic programming and inspired by the work on BDI architectures [20]. Although attempts to define methodologies for BDI systems started in the mid-90s [64], introducing this new language has raised the need to define specific methodological approaches in the AOSE community. Indeed, the opportunity to focus on new techniques like the plan and belief models seems evident. For example, in WOA 2013, a semantic approach and a set of guidelines have been presented to identify goals to implement BDI agents [82].

Shifting from goals as requirements towards run-time models (see the model@run-time manifest [17]) was a fundamental step in designing and creating requirement-aware agents able to reason about their actions and change their behaviour at run-time. This may drive their adaptation to optimise performance, ensure reliability, and maintain security. In WOA 2015, a Proactive Means-end Reasoning (PMR) agent has been presented [36]. The main intuition was to decouple goals and tasks, allowing the agents to establish a link at run-time. The strength of this approach has been discussed in a methodology [41] presented at WOA 2017. It fosters collaborations between analysts and agents whereas the role played by the agents consists in addressing the goals provided/changed at run-time. Conversely, the analyst's role is to collect data during the system execution, creating a better understanding of the conditions and thus continuously designing new goals/capabilities for the agent.

3.2 Situatedness

Agents perceive and act in an environment that may include a communication channel. The environment may also be a virtual or physical place where the effects of agents' actions are observed and evaluated. Therefore Situatedness was one of the first principles of agent conceptualisation. Indeed, agent-oriented methodologies have always emphasised the need to model the environment as a specific abstraction. A common implementation of situatedness in Multi-Agent Systems (MAS) involves agents sensitive to environmental change.

In the earlier phases, the design of the agents' environment suffered because frameworks like JADE and FIPA-OS did not include specific entities for implementing it. Indeed, methodologies like MAS-Common KADS [1] focussed on the domain of interest from the agent's point of view, i.e., providing instruments to model the knowledge the agents need to act in the environment and achieve their goals. PASSI [43] uses a similar approach but emphasises the need to conceptualise the elements of the domain by using an ontology. In particular, the Ontology Description (O.D.) is a diagram that adopts a UML profile and a graphical notation for describing a semantic formalisation of the domain. Inspired by the FIPA specification of ontology, it provides three main elements: Concepts are the elements of interest in a domain; Predicates are their properties, whereas Actions represent how the agent can manipulate these domain elements. In WOA 2013, [82] extended the use of O.D. diagrams, suggesting some guidelines for using an ontology to represent the domain of interests to address possible ambiguities and incompleteness. In addition, that work proposes a process to identify agents' goals directly from the ontology.

As we anticipated, agents must be programmed to respond to environmental events. In WOA 2010, the TuCSoN coordination model [71] has been presented to support the development of information-oriented applications. It provides two abstractions: a programmable coordination model based on tuples and a computational activity called reaction. TuCSoN addresses situated, open and distributed MASs by relying on mediating abstractions (such as ACC, transducers, and tuple centres) to decouple interactions between individual components (agents and probes). Additionally, an asynchronous event-driven communication model handles common system distribution issues. In WOA 2014, the latest research results added abstractions for requirement analysis, modelling architectural components, and programming situatedness-related aspects via a coordination API [66].

In WOA 2000, the SODA methodology [73] explicitly included an environment model where logical and physical resources are mapped onto infrastructure classes and associated with topological abstractions. The main idea behind this approach is that the structure of an environment may be shifted into a run-time model that the agent can use to reason on and to make decisions.

Later on, situatedness was approached by adopting a nature-inspired perspective [2]. In WOA 2008 and 2010, Zambonelli et al. in [93,95] list situatedness and adaptivity as fundamental requirements for pervasive systems and argue that the best architecture to implement a pervasive system with such features

should be a nature-inspired ecosystem where a set of spatially situated entities interact following some natural laws enforced by their living environment, and adaptively self-organize their interactions in synergy with the environment itself.

3.3 Sociality

When adopting the multi-agent perspective, agents are regarded as social entities; although each agent may have its own agenda, they are supposed to work together to achieve their goals. Cooperation is a fundamental concept that sets the MAS paradigm apart from other related disciplines, such as distributed computing and object-oriented systems. Additionally, the idea that agent organisations can produce emergent behaviour has been evident in developing Multi-Agent Systems from the beginning.

To effectively organise autonomous entities into cooperating groups, the AOSE community recognised the need for modelling techniques to manage these systems' complexity and guide the design process. This ensures that the resulting systems are reliable, maintainable, and extensible. In [81], the authors have investigated the use of Multi-Agent Systems for traffic simulation, control, and management and provide a structured overview of the most relevant research, classifying the studies according to three key criteria: The purpose of the agents, the applied artificial intelligence techniques, and the behaviours of the agents. They thus support the idea that centralized and static approaches are insufficient to manage complex and dynamic environments such as urban transportation networks, reinforcing the notion that cooperative interactions between agents exchanging information enable self-organizing mechanisms that allow agents to adapt to changing conditions, a key principle of agent societies.

A remarkable contribution to conceptualising agents' social structures comes from Moise+ [62], which introduces three dimensions: structural, functional, and deontic. The structural dimension determines the system's organisational structure regarding groups, roles, and the functional relationships between those roles. Often, the relationship between agents and the group's roles is established at run-time, as in [30]. The functional dimension focuses on how various groups of agents are expected to work together to achieve shared goals. Finally, the deontic dimension relates agents/roles to rules and constraints. It sets the standards for acceptable behaviour within the system and outlines consequences for violations.

This approach fits the need for an AOSE perspective on the social aspects of agency. In the following, we will discuss contributions presented at WOA events about sociality, clustering them according to the three dimensions introduced above.

Structural Dimension. Focussing on the contributions presented at WOA on the structural dimension of a Multi-Agent System, a significant contribution comes from SODA [73] (Societies in Open and Distributed Agent Spaces), which mainly focuses on inter-agent issues, such as engineering societies and infrastructures for Multi-Agent Systems [24]. It uses a social model to depict

groups regarding social roles, tasks, permissions, and interaction rules. Societies in SODA are designed around coordination media, which are abstractions provided by coordination models for coordinating multi-component systems.

Some years later, ASPECS (Agent-oriented Software Process for Engineering Complex Systems) [32] proposed a holistic view instead of the classical agent organisation. This methodology deals with how to design agent societies under a holonic perspective (HMAS). The idea is that holons are agents that may be (recursively) composed of agents for developing complex software systems, looking at the problem domain with different levels of granularity. The advantage is that it provides instruments for the design of open, dynamic, and complex systems. During WOA 2013, in [82], the authors presented a methodology for designing norm-governed holonic MAS. Its metamodel was created by combining elements of ASPECS with all those concepts from Moise+, which are useful for modelling a MAS from a normative point of view. Another contribution was presented at WOA 2020; it proposed an approach for automatically defining MOISE+ organisations starting from the Business Process Modeling Notation (BPMN) specification of a business process [37].

Functional Dimension. Earlier attempts were made to adapt existing modelling techniques and notations from the Object-Oriented side, which achieved considerable maturity and widespread acceptance. A great effort, in particular, has been focused on defining interaction protocols [5] and coordination models [24]. The FIPA organisation and the OMG Agent Work Group followed and suggested this trend, supporting the exploration of UML use and the development of several extension proposals for the agent domain.

FIPA's activity comprises several Technical Committees (TCs), each responsible for producing specifications that become part of the FIPA standards. The activities of three active TCs have been presented at WOA 2003 in [34]. The first was the "Interaction Protocols TC", which aimed to develop flexible conversation policies between agents and agent-based systems. The second one was the "Methodology TC" which focussed on identifying a standard design methodology for Multi-Agent Systems (MASs) that could address various needs. The adopted approach involved creating a meta-methodology that could be instantiated in specific methodologies for different problems. This trend has been adopted in several subsequent research projects involving the international community. Finally, the "Modeling TC" worked on defining specifications for a future FIPA agent-based unified modelling language starting from the experiences done with UML and AUML.

The most notable example of accommodating UML for the distinctive requirements of agents was Agent UML or AUML [23]. AUML is primarily known for the layered approach to representing agent interaction protocols, i.e. the protocol diagram (derived from the UML sequence diagram). AUML was also adopted in a paper presented at WOA 2003 [40] for depicting the deployment details of an agent system; the paper also deals with other implementation

challenges of developing agents like agents-interactions and pattern-reuse that have been deepened in many other papers in the following years.

Tropos made another significant contribution to the functional dimension of modelling agents. This design methodology focuses on collaborations between humans and agents and between different agents. This approach makes it possible to model “delegation” relationships that reveal the dependencies among the entities involved in the contest. This type of analysis allows for creating intentional, collaborative contexts in which agents interact with a specific purpose in mind. In fact, at WOA 2008, the authors of [44] explore several fundamental features of the Tropos methodology that play a crucial role in supporting the development and runtime reconfiguration of socio-technical systems (STSs). Specifically, they address two critical design challenges, risk analysis and location variability, and discuss how they are integrated and used to support the designer in choosing the best design alternative. The paper also presents a generic framework to develop self-reconfigurable STSs.

Deontic Dimension. Historically, ADELFE [79] was one of the first and most known methodologies for self-organising agents with a specific focus on deontic aspects of the MAS. It includes a cooperation view where the designer can design local rules to make the agent locally cooperative. However, in ADELFE, agents’ cooperation is not directly modelled. Being cooperative means that an agent can recognise Non-Cooperative Situations (NCS), i.e. states that it judges opposing to an ideal cooperation.

Many papers have been presented at WOA events dealing with related aspects of sociality, such as delegation, collaboration, cooperation, etc. We leave the discussion of these contributions to other chapters of this book. We note that few agent-oriented methodologies in the literature deal explicitly with deontic abstraction. This open point may trigger future work in the AOSE research area.

Surely, an original point of view about agent’s organisations has been presented at WOA 2008 and 2010 [93,95]. In the papers, the authors identify the need for innovative paradigms and frameworks that could allow the effective deployment and execution of pervasive computing services. They propose to model the normative part of the system by considering that the components interact and evolve based on a limited set of spatial ‘laws of nature’, and they introduce a reference architecture to frame these concepts consistently.

4 Agent Methodologies: Metamodel, Fragments and Notation

From the beginning of the WOA’s history, we have found interesting contributions in the design of agents and agent-based robotic systems. During WOA 2001, the authors of [28] presented an initial study of a methodological approach towards designing a multi-agent robot architecture, which is an initial step towards defining the PASSI methodology.

In the same year, the TROPOS methodology was introduced in [78], and the next year, an application of this methodology was presented in [61]. This paper illustrates the TROPOS early requirements analysis phase applied to the Ice Co case study; it required the design of an agent system to enable Trentino's industries to easily access some online logistic services. As introduced in this paper, the Tropos methodology encompasses five stages of software development. Initially, early requirements analysis focuses on understanding organizational issues by creating a model that outlines the actors and their interdependencies. These actors have goals that are more effectively achieved through shared knowledge and mutual dependencies. Specifically, dependencies involve a depender, a dependee, and a dependum (which could be goals, soft goals, plans, or resources), with the depender entrusting the dependee to fulfill a particular dependum. Subsequently, late requirements analysis characterizes the intended system within its operational context, detailing its functions and attributes. The system is depicted as a set of actors with various social dependencies within their environment. In the architectural design phase, subsystems (as actors) and their interconnections (as actor dependencies) define the system's overarching structure. The detailed design then elaborates on each subsystem using a selection of AUML diagrams, specifying inputs, outputs, controls, and other pertinent details. Lastly, the implementation phase actualizes the system following the detailed design specifications.

A different research stream presented at various WOA editions regarded the adoption of metamodels in system design and the composition of design methodologies. This research finds its motivation in considerations that arose from the beginning of software engineering research and practice.

In fact, since the late 1960s, the field of software production has recognized that a disciplined approach is necessary to handle complex systems. This involves specific design activities to analyze the problem domain, identify potential goals and requirements, and convert them into software. Tying a design methodology to the programming language is a good practice.

One possible way to achieve that is through the concept of a metamodel. A metamodel is a high-abstraction model that describes a system's structure and behaviour. It provides a framework for understanding the system and its components and guides the design process.

In the initial stages of a project, it is important to conduct a theoretical analysis to identify the application domain in detail, decide which technologies to use, and then apply the appropriate design methodology. This can be done by creating a metamodel that outlines the basic principles of the design process and abstracts the agent domain. MOF (Meta-Object Facility)¹ is a metamodeling language used to define the structure and behaviour of other modelling languages, such as UML (Unified Modeling Language). UML is a modelling language commonly used in software engineering to represent the design of a system. It is based on a metamodel that includes elements such as classes, attributes,

¹ <https://www.omg.org/spec/MOF/2.5.1/About-MOF>.

and instances. Both MOF and UML are used in many engineering approaches to software development.

MOF and UML are metamodeling languages used to represent and study the world in a specific syntax. The elements in MOF and UML, such as classes, attributes, and instances, are also linked to the problem domain being studied. For instance, in the context of robotic systems, MOF and UML can be used to model the system-to-be as it happens in classical software engineering approaches. Moreover, these languages also allow the drawing of agent-related models representing the knowledge, the decision-making processes of the robot, and its actions.

At the end of 2004, a great European initiative started and actively involved the WOA community: the AgentLink Network of Excellence. Much effort has been spent during the Agentlink meetings on the AOSE topics. A specific working group has been created, giving place to a vibrant and productive exchange of experiences and several scientific collaborations. Several members of the WOA community actively participated in this initiative and brought back into the WOA events the results of that work, enriching them with new contributions. A proposal of a unifying MAS metamodel was one of the main results of the numerous meetings held around Europe during the AgentLink events [15].

In the following subsections, we will discuss details of the research presented at WOA dealing with metamodels, method fragments, the composition of ad-hoc design methodologies and modelling notations.

4.1 Metamodel

A metamodel is a model that describes the structure of all the elements that should be designed when following a specific methodology. In [45], the authors propose an integrated agent-oriented metamodel that merges the best aspects of four relevant AOSE methodologies. This metamodel is useful because it helps fill the gap between the methodology's conceptual level and the system's implementation level, resulting in more consistent and well-designed agent systems. Filling the gap refers to the process of bridging the differences and integrating the concepts and methodologies of AOSE methodologies and MAS infrastructures. This complex task involves comparing and relating the concepts, providing methodological guidelines, and introducing new concepts to make the integration successful. The ultimate goal is to create a seamless transition between the design and implementation phases, allowing for a more efficient and effective development process for agent-based software.

The core fragments of the metamodel were identified by first defining the requirements for the metamodel and then selecting fragments that fulfilled those requirements. The fragments were then analyzed and refined to provide better integration. A glossary of relevant terms was also created to aid in identifying synonyms and conflicts. Based on this work, a first version of the metamodel was then defined.

An interesting approach to using meta-model is adopted in [16] where the elements and parameters of an evolutionary Multi-Agent System are modelled

at the meta-level and instantiated at runtime. The authors adopt this approach to observe how the behavioural parameters of agents adapt under evolutionary pressure at runtime.

4.2 The Method Fragment and the Repository

After the AgentLink initiative, in 2010–2014, the community of researchers working on AOSE continued to hold meetings often co-located with scientific events in Europe. The work also involved the FIPA association, which, at the time, was looking for a methodological reference specifically conceived to develop FIPA-based MASs. The result of this work was the conception of a new methodological approach where existing AOSE methodologies contributed to constructing a repository of pieces of process (called method fragments) that could be assembled to build ad-hoc design processes. The standardization perspective pursued by the FIPA association helped achieve the result of a standard approach for the documentation of the existing methodologies that enabled the identification of method fragments, the construction of a repository and the assembly of new, ad hoc processes. The WOA community has delivered several papers on these topics. The final result of this long effort was a book collecting most of the well-known agent-oriented design processes and describing them according to the FIPA standard description approach [33], again the WOA community largely contributed to that and fruitfully linked that effort to the WOA events.

A method fragment is a portion of a design process composed of two main parts: the process and the product. It is stored in a repository and can be used to construct new design processes or retrieve specific fragments for reuse. A method fragment provides a reusable piece of a design process that method engineers can use to create new methodologies. Method engineers select and assemble method fragments to construct a new process tailored to an organization's specific needs and capabilities. In [89], the authors utilized method fragments by extracting, defining, and standardizing portions of existing design processes and storing them in a repository. This database repository categorizes fragments based on process metamodel elements central to agent-systems design: process role, phase/activity, work product, and MAS Metamodel element. The fragments are described using SPEM diagrams and text documents, illustrating their use in various activities and phases of the design process. This approach allows for easy retrieval and assembly of fragments to construct new, situation-specific design processes, facilitating the reuse and adaptation of well-defined process components in Multi-Agent Systems development.

Based on this and again in [89], the authors propose to use method engineering to design, construct, and adapt methods for information systems development. This involves extracting, defining, standardizing, and storing method fragments from existing processes in a repository. The method engineer selects and assembles the appropriate fragments to create a new process. This approach allows for reuse and customization of existing methods.

The repository includes metadata such as Fragment ID, Fragment Name, File Name, and Phase ID. These are managed using a relational data structure,

with relationships such as Fragment (ID, FragName, FileName, ID Phase) and Phase (ID, PAName, PADescription). Other elements in the repository may also have metadata associated with them. Creating a repository of fragments is important because it allows for easy access and retrieval of standardized method fragments, which can then be used to compose new design processes. This saves time and effort in the design process and ensures consistency and efficiency in using method fragments.

A method fragment consists of composition guidelines, aspects of the fragment, and dependency relationships. It is also important to consider the context and problem behind the fragment and the specific issues it addresses. Moreover, it comprises several elements that describe a portion of a design process. These elements include activities, process roles, and work products. Activities refer to a portion of work performed by a process role and can be further broken down into smaller parts called steps. Process roles are responsible for performing the activities and can be assigned to one or more individuals. Work products are the outputs of an activity and can include diagrams or text documents.

In addition to these core elements, a method fragment includes composition guidelines, aspects of the fragment, and dependency relationships. Composition guidelines describe the context or problem the fragment intends to address, which helps with its reuse in similar situations. Aspects of the fragment provide additional information about the fragment, such as the preferred implementation platform or application area. Dependency relationships can be used to identify other fragments strongly related to the one being considered.

A method fragment is a comprehensive representation of a portion of a design process, including the necessary elements and additional information to facilitate its use and retrieval from a repository.

A link with the previously cited FIPA activity on design methodology can be found at WOA 2010, where [69] presents an experimental evaluation of the methodology documentation template proposed by the IEEE FIPA Design Process Documentation and Fragmentation working group. This template documents the three fundamental elements of the methodology (metamodel, notation, and process) and adopts OMG's standard Software Process Engineering Metamodel (SPEM). The experiment is performed by documenting the SODA process.

Interestingly, the authors propose evaluating and comparing SPEM 2.0 (Software Process Engineering Metamodel) with SPEM 1.0 in the same case study [68]. In the paper, the authors examined the applicability of SPEM 2.0 to the SODA methodology. Their goal was to determine whether SPEM 2.0 addressed the limitations and expressiveness issues that had arisen in SPEM 1.0, particularly related to the readability of UML diagrams in the context of complex Agent-Oriented methodologies and the absence of suitable ad-hoc entities. The findings indicate that SPEM 2.0 effectively addresses some of these limitations by clearly separating Method Content and Processes. Additionally, it incorporates capability patterns and enables the expression of relationships between Work Product states and the Activities responsible for changes in those states.

4.3 Method Engineering and Creating a Methodology

In [56], method engineering is employed to construct tailored development processes by integrating various method fragments. Two principal approaches are utilized: metamodel-driven and development process-driven approaches. The metamodel-driven approach emphasizes using a MAS metamodel to guide the methodology construction. This approach requires selecting or defining a MAS metamodel that is appropriate for the specific problem and application domain. Subsequently, the elements that comprise the metamodel of the MAS under development are identified. Method fragments, which are capable of producing the identified metamodel elements, are then selected. The development process is characterized by the execution order of these method fragments, based on the relationships among the metamodel elements. This ensures that the resulting methodology comprehensively covers the MAS metamodel for the given problem and application domain, providing a flexible framework for defining methodologies and metamodels for MAS development. Conversely, the development process-driven approach focuses on instantiating a complete software development process encompassing all MAS development phases. In this approach, a suitable software development process is either chosen or defined, tailored to the specific problem and application domain. Each phase of the development process is instantiated by selecting appropriate method fragments from existing agent-oriented methodologies or by defining new fragments as needed. Fragments are integrated by identifying and defining dependencies among the work products produced in each phase. This ensures that the work products from one phase serve as the inputs for the subsequent phase, creating a coherent and seamless development process.

A tool to support development activities is crucial, whether the designer is using a methodology to develop a software product or is creating a new one.

In [25], the authors present MetaMeth, a tool for process definition and execution in software engineering. It is a Computer Aided Method Engineering (CAME) and Computer Aided Process Engineering (CAPE) tool at the same time, meaning it supports the construction of new methods through reuse and also allows for the management and execution of processes through a workflow engine, agents, and an expert system. It also has a code generator for implementation and supports design patterns for reuse. The main innovation of MetaMeth is combining CAME and CAPE functionalities, allowing method engineers to not only store and retrieve reusable method fragments but also to orchestrate all Computer Aided Software Engineering (CASE) services and design a system. It also integrates a reuse technique based on design patterns and has a code generator that uses an MDA approach to transform the design view into an implementation view and, finally, a code view. Additionally, it supports distributed design and automatic composition of work products. In MetaMeth, process definition and execution are separated using a multi-agent system. Process definition is done through a graphical workflow editor called JaWE, while process execution is done through a separate module that uses a Multi-Agent System. This allows for a clear separation of concerns and more efficient process management.

The objective of automatic composition is to automatically modify, create, or update work products related to a specific activity in the design process. This is done using auto-composition, notation interpretation, and semantic validation services offered by activity agents.

Another contribution to the stream of the composition of ad-hoc methodologies and related tools comes from [46]; in this paper presented at WOA 2011, the authors introduce PROCE (PROcess Composition and execution Environment), an agent-based CAME and CASE tool which supports the design of Multi-Agent Systems as prescribed by the Organization-based Multi-Agent System Engineering (O-MASE) methodology [58]. The tool also comprises a fragment repository inspired by the FIPA fragment specification [33]. The tool allows the development of agent systems that are compatible with the JADE platform [11].

4.4 Agent-Based Notations for Designing MAS

Notation, in general, in the metamodeling process has a very important role because it helps to define the structure and syntax of the methodology being developed. In [12], authors evidenced a gap between agent technology and widely accepted industrial technologies. They proposed integrating Multi-Agent Systems (MAS) into real-world applications. Indeed, despite significant advancements in agent technology, its industrial and commercial applications remain limited. The authors aim to leverage the Unified Modeling Language (UML) by extending it to accommodate the concepts of software agents and MAS through Agent UML (AUML). This integration proposes MAS as an incremental extension to existing, proven industrial practices. The practical application of AUML in the WINK project, which focuses on data management and system updates, evaluated the suitability and effectiveness of AUML in a real-world scenario and identified areas for refinement and improvement in the modelling language. The authors conclude that there is a lack of development tools to support the graphical representation and code translation of AUML, and further work is needed to clarify the semantics of dependencies and cooperation between agents.

In [40], the authors argue that existing agent-oriented modelling languages, like Agent UML (AUML), need enhancements to fully capture aspects related to representing interactions between agents and their internal architecture, deployment of MAS, and pattern reuse. They propose an extended AUML that includes detailed deployment diagrams and interaction representations, aiming to provide developers with precise tools to map design models to implementation artefacts. They revealed the inadequacy of current modelling languages in addressing the implementation-level details of MAS. While significant efforts have been made to define metaphors and diagrammatic notations for MAS, these often fall short in practical implementation scenarios. Specifically, the transition from design to implementation lacks sufficient support, making it difficult to manage the complexity and heterogeneity of MAS.

The notation is also used during process execution to ensure the methodology is followed correctly and to identify any errors or inconsistencies. The MetaMeth tool allows for introducing syntactic and semantic rules to ensure

that the notation is used correctly. Additionally, the tool has a graphical notation feature that allows for creating complex documents using diagrams, text, tables, and more. The work done during the realization of the Metameth tool also concerned researching in the notation field. The question was: *What are the most efficient and representative elements that can be used for a notation suitable for an agent-based design methodology?*

The main objective of this research is to propose notations and models for representing social structures and relationships in the context of organizational and agent-based systems. The UML-based notation in [35] is designed to represent organizational elements in MASs, whereas other methods may not have been specifically tailored for this purpose. Additionally, the proposed notation allows for representing all the elements of the Moise+ organizational model, whereas other methods may only represent certain aspects of the organization.

Indeed, at the time, Moise was not adequately supported by a well-defined methodological approach. Creating a specific notation for Moise makes it easier to represent and explain the organizational model to stakeholders involved in the design process, making it more readable and understandable. This also allows for the involvement of stakeholders in the development process, improving the flexibility of conventional development approaches. The proposed graphical notation allows the representation of organizations in the Moise+ organizational model. It includes three types of diagrams: structural, functional, and normative. It is designed to be easily understandable by stakeholders and can be used to support different approaches. However, it is not as comprehensive as other notations, such as the one presented in [76], including behavioural diagrams.

A complementary effort is made in [82] where the authors propose using ontology to describe the problem domain to better understand the problem and identify goals. This can lead to improvement in the goal identification process and potentially address any inconsistencies or deficiencies. Here, the authors propose a formalization for the Problem Ontology Description activity, which aims to describe the intentional behaviours of problems addressed with agent-oriented technologies. The innovation consists of introducing new elements inspired by the FIPA agent ontology to explicitly model intentional behaviours and identify the goals the solution of the problem has to satisfy. The paper presents a formalization for the Problem Ontology Description activity, which aims to describe the intentional behaviours of problems addressed with agent-oriented technologies. The innovation proposed is the introduction of new elements inspired by the FIPA agent ontology to explicitly model intentional behaviours and identify the goals the problem's solution has to satisfy.

The problem ontology can be used to improve the analyst's understanding of the problem by providing a deeper knowledge of the problem domain and identifying goals. This can be achieved through reasoning on the elements and relationships within the ontology and by identifying patterns that can provide useful information about the goals of the problem. The ontology can also help to address any inconsistencies or deficiencies in the problem specification. The ontology can also help eliminate redundancies and ambiguities in the problem

specification, making it easier for stakeholders to understand and communicate their knowledge. Additionally, the ontology can assist in identifying the artefacts that will compose the environment, providing a clear framework for the design process.

One of the difficulties in defining the system ontology is correctly catching the key elements of the problem description and mapping them to corresponding elements of the ontology. At WOA 2014, [39] proposed GIMT (Goal Identification and Modeling Tool), a CASE tool for supporting ontology building and goal modelling. GIMT allows users to write the problem statement and extract significant elements to build the system's ontology. The ontology is then analysed to identify the system's goals. This is a first step in the construction of the problem ontology. By adding concepts from the solution domain, such an ontology can be evolved to support agents' communications, thus obtaining a rigorous foundation for both agents' reasoning and semantic communications.

Over the years, the need to represent and manage knowledge in Multi-Agent Systems operating in dynamic and unpredictable environments was addressed, also in mission-critical and sensitive applications like emergency management [38]. First, modern software systems must consist of increasingly autonomous and self-adaptive entities capable of performing tasks that were not always predictable in the design phase. These systems must interact and collaborate with humans as if they were a team. They must exchange knowledge, decide whether to perform actions independently or delegate them to others and adapt to changes in the environment in real time. The representation of knowledge is crucial for decision-making. Agents must be able to update and manage their knowledge while the system runs, especially in operational areas that change during runtime. In [29], the Belief-Desire-Intention (BDI) model has been used, and programming languages such as Jason enable the mapping of intentions and events at runtime, the selection of plans from a library of plans to achieve a goal even in situations that are not fully known at the design stage, and the exploration of knowledge representation and management in Multi-Agent Systems using the Jason programming language, the CArTAgO framework and the OWL ontology. The focus is on creating an agent memory system that is adaptive and autonomous in dynamic environments. The authors propose a centralized agent called Jino that manages episodic and semantic memory and improves the efficiency of agents' decision-making. The methodology enables agents to adapt to changes in real time, ensuring effective knowledge management through a centralized approach.

The development of agent-based methodologies in recent years has shown the importance of using modular and flexible methods, as supported by fragmented methods and CAME/CASE tools, which provide a solid foundation for developing increasingly complex and adaptive systems. Looking to the future, the continued integration of methods, methodologies and tools promises to further advance the application of agents in industrial contexts and improve adaptability and collaboration in dynamic environments.

5 The Future of AOSE

The future of Multi-Agent Systems still holds immense potential for revolutionizing industries and transforming how we interact with technology. In health-care, agent-based systems can optimize patient care, manage clinical trials, and facilitate personalized medicine. In smart cities, agents can enable intelligent infrastructure management, public safety, and resource allocation in urban environments. Self-driving vehicles will become more prevalent, while agents can optimize routes, traffic flow, and logistics in transportation. Manufacturing will see improved production processes, managed supply chains, and enhanced product quality through the application of software engineering methods.

As researchers and developers, it's crucial to address the challenges and opportunities that lie ahead. There are significant challenges to overcome. One such hurdle is the need to design and manage complex systems comprising multiple autonomous and interacting agents that can adapt to changing environments, learn from experiences, and make decisions without human intervention. This requires a deep understanding of how agents establish trust with each other and maintain reputations in distributed systems.

Another crucial area of focus is the IoT and edge computing. As the number of connected devices grows, edge computing will play a vital role in processing data closer to the source, reducing latency, and improving real-time interactions between agents. Researchers must study how agents can establish trust with each other and maintain reputations in distributed systems, ensuring reliable interactions and cooperation. The WOA community has always been relevant to the research about AOSE and, consequently, has considered the future of this area of research. A relevant contribution to the discussion comes from [94], where the author states that AOSE addresses the challenge of engineering large-scale systems involving goal-oriented entities. These entities often have conflicting goals and involve multiple stakeholders. This problem relates to the challenges faced by the IoT, where accommodating services and managing many goals are crucial. While the concept of goal-oriented groups draws inspiration from AOSE, IoT necessitates specific concepts and abstractions that AOSE, in its general terms, does not directly cover.

Another contact point between IoT and AOSE is in the research for adaptivity that AOSE has pursued as a branch of the general area of software engineering. This addresses the challenges posed by dynamic and unpredictable operational environments. These dynamics also impact IoT systems, where ensuring continuous functionality involves embedding close control loops (similar to those in self-adaptive systems research). These loops continuously monitor system activities and the environment, allowing for eventual corrective actions. The author in [94] introduces and outlines some key conceptual abstractions related to the IoT domain and suggests an initial step toward establishing a broader discipline for engineering IoT systems and applications starting from the AOSE background.

The convergence of physical and computational systems will lead to more complex, adaptive, and resilient systems that require AOSE's capabilities. Com-

plex systems will require multiple agents to interact with each other to achieve common goals, making Multi-Agent Systems a crucial aspect of AOSE's future. Researchers are currently investigating how multiple agents can learn from each other and their environment to achieve common goals or optimize individual performance. The role of AOSE is to provide models and methodologies for developing and simulating complex systems using autonomous agents, enabling a better understanding of dynamics, behaviour, and decision-making processes. The big challenge here is protecting autonomous agents from cyber threats and ensuring data integrity and system stability.

Developing seamless interfaces between humans and autonomous agents for effective collaboration is another key trend worth exploring. In this context, integrating artificial intelligence (AI) and machine learning (ML) into agents will enable more sophisticated decision-making, planning, and problem-solving capabilities. Now, the recent hype around Large Language Models (LLMs) warrants a dedicated discussion, suggesting a fascinating relationship between AOSE and LLMs. Conversational interfaces powered by LLMs can enable humans to interact with autonomous agents through natural language processing (NLP) and text-to-speech synthesis. This integration can facilitate more sophisticated coordination and negotiation among autonomous agents, leading to improved decision-making and system performance. The role of AOSE is to leverage LLMs to enable agents to better understand natural language, which is essential for effective communication and decision-making in complex systems. However, this also raises concerns about robustness against adversarial attacks and biases.

Cognitive architectures can couple symbolic reasoning to amplify agents' power in analyzing complex situations, learning from experience, reasoning about complex situations, and adapting to changing environments. For instance, LLMs can create knowledge graphs representing complex relationships between entities, concepts, and actions. This enables agents to reason about the world more effectively and make informed decisions. Interpretability and explainability become essential in this context: methods are required to interpret the output of LLM-based decision-making systems and explain decisions made by autonomous agents using LLMs.

Explainability and interpretability are also fundamental in human-agent interaction, especially when humans have to interact and collaborate with machines (such as computers, robots, or other devices) in environments that are not fully known a priori. In these cases, the challenge is to use agents not only as a design paradigm but also as a suitable technological approach. In human-robot interaction, for example, agents can solve complex problems related to ethical issues. The ability of an agent to reason and deliberate based on its own knowledge can be a crucial factor in building the right agent architecture. Indeed, an appropriate ontological description is normally needed to give an agent the means to link actions to the elements in the environment and to the goal. For instance, in a soccer scenario, the goal may be to pass the ball to a team member, and in this case, it may be possible for an agent to commit the action "to throw a ball at someone". If, on the other hand, the scenario is a small child's

entertainment, throwing the ball could even be dangerous. This makes sense, and it is necessary to adequately describe the knowledge base, especially regarding ethical norms. In this context, it might be interesting to investigate using knowledge graph technologies.

In this context, the possibility of endowing an agent with self-analysis and self-disclosure abilities that lead to a clear and reasoned explanation of how and why an action is performed can be explored. Two preliminary papers have already been presented at WOA in the last two years [90,91]. Additionally, LLMs, text-to-speech techniques, and cognitive architectures could prove to be fundamental and enriching research elements in the agent world.

Another potential use of agents, particularly in robotics, is their application in designing and implementing complex robotic systems that rely on ROS and Gazebo. ROS and Gazebo have become almost a de facto standard in robot programming. The potential of agents to handle the complexity of systems, combined with the strengths of ROS and Gazebo from an implementation perspective, could lead to significant advancements in AOSE in the future.

References

1. Aguilar, J., Cerrada, M., Mousalli, G., Rivas, F., Hidrobo, F.: A multiagent model for intelligent distributed control systems. In: Knowledge-Based Intelligent Information and Engineering Systems: 9th International Conference, KES 2005, Melbourne, Australia, 14–16 September 2005, Proceedings, Part I. Lecture Notes in Artificial Intelligence, vol. 3681, pp. 191–197. Springer (2005). https://doi.org/10.1007/11552413_28
2. Aguzzi, G., Casadei, R., Pianini, D., Viroli, M.: Self-organisation with aggregate computing: a reflection under the lenses of multi-agent systems engineering. In: Mascardi and Omicini [67]
3. Armano, G., Paoli, F.D., Omicini, A., Vargiu, E. (eds.): WOA 2003 – 4th Workshop “From Objects to Agents”. Pitagora Editrice Bologna, Villasimius, CA, Italy (2003). <http://giuseppezizzari.github.io/WOA-proceedings-archive/woa-2003.html>
4. Baldoni, M., Baroglio, C., Bergenti, F., Garro, A. (eds.): WOA 2013 – 14th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 1099. Turin, Italy (2013). <http://ceur-ws.org/Vol-1099/>
5. Baldoni, M., Baroglio, C., Micalizio, R.: Interaction protocols: from AURL to social commitments, from artifacts to BSPL. In: Mascardi and Omicini [67]
6. Baldoni, M., Boccalatte, A., De Paoli, F., Martelli, M., Mascardi, V. (eds.): WOA 2007 – 8th Workshop “From Objects to Agents”. Seneca Edizioni Torino, Genova, Italy (2007). <http://woa07.disi.unige.it/ProceedingsWOA2007.zip>
7. Baldoni, M., Cossentino, M., De Paoli, F., Seidita, V. (eds.): WOA 2008 – 9th Workshop “From Objects to Agents”. Seneca Edizioni Torino, Palermo, Italy (2008). <http://www.pa.icar.cnr.it/woa08/materiali/Proceedings.pdf>
8. Baldoni, M., De Paoli, F., Martelli, A., Omicini, A. (eds.): WOA 2004 – 5th Workshop “From Objects to Agents”. Pitagora Editrice Bologna, Torino, Italy (2004). <http://lia.deis.unibo.it/books/woa2004/atti.pdf>
9. Barile, F., Caso, A., Rossi, S.: Group recommendation for smart applications: a multi-agent view of the problem. In: Santoro and Bergenti [85], pp. 12–17. <http://ceur-ws.org/Vol-1260/paper7.pdf>

10. Baxter, J.L., Burke, E.K., Garibaldi, J.M., Norman, M.: Multi-robot search and rescue: a potential field based approach. In: *Autonomous Robots and Agents*, 9–16 (2007). https://doi.org/10.1007/978-3-540-73424-6_2
11. Bellifemine, F., Poggi, A., Rimassa, G.: Developing multi-agent systems with JADE. In: *Intelligent Agents VII: Agent Theories Architectures and Languages*, 7th International Workshop, ATAL 2000 Boston, MA, USA, 7–9 July 2000, Proceedings, pp. 89–103. Springer (2001). https://doi.org/10.1007/3-540-44631-1_7
12. Bergamaschi, S., Gelati, G., Guerra, F., Vincini, M.: Experiencing AUML for the WINK multi-agent system. In: Armano et al. [3], pp. 148–154. <http://giuseppevizzari.github.io/WOA-proceedings-archive/pdfs/woa2003/20.pdf>
13. Bergenti, F. (ed.): WOA 2009 – 10th Workshop “From Objects to Agents”. Seneca Edizioni Torino, Parma, Italy (2009). <http://www.ailab.unipr.it/woa09/papers/>
14. Bergenti, F., Iotti, E., Poggi, A.: Outline of a formalization of JADE multi-agent systems. In: Di Napoli et al. [50], pp. 123–128. <http://ceur-ws.org/Vol-1382/paper19.pdf>
15. Bernon, C., Cossentino, M., Pavón, J.: Agent-oriented software engineering. *Knowl. Eng. Rev.* **20**(2), 99–116 (2005). <https://doi.org/10.1017/S0269888905000421>
16. Bertolotti, F., Roman, S.: The evolution of risk sensitivity in a sustainability game: an agent-based model. In: Ferrando and Mascardi [54], pp. 101–115. <http://ceur-ws.org/Vol-3261/paper8.pdf>
17. Blair, G., Bencomo, N., France, R.B.: Models@run.time. *Computer* **42**(10), 22–27 (2009). <https://doi.org/10.1109/MC.2009.326>
18. Booch, G., Jacobson, I., Rumbaugh, J., et al.: The unified modeling language. *Unix Rev.* **14**(13), 5 (1996)
19. Bordini, R.H., Hübner, J.F.: BDI agent programming in AgentSpeak using *jason*. In: *Computational Logic in Multi-Agent Systems: 6th International Workshop, CLIMA VI*, London, UK, June 2005, Revised Selected and Invited Papers, pp. 143–164. Springer (2005). https://doi.org/10.1007/11750734_9
20. Bratman, M.E., Israel, D.J., Pollack, M.E.: Plans and resource-bounded practical reasoning. *Comput. Intell.* **4**(3), 349–355 (1988). <https://doi.org/10.1111/j.1467-8640.1988.tb00284.x>
21. Bresciani, P., Giorgini, P., Giunchiglia, F., Mylopoulos, J., Perini, A.: Tropos: an agent-oriented software development methodology. *Auton. Agent. Multi-Agent Syst.* **3**(8), 203–236 (2004). <https://doi.org/10.1023/B:AGNT.0000018806.20944.ef>
22. Brinkkemper, S.: Method engineering: engineering of information systems development methods and tools. *Inf. Softw. Technol.* **38**(4), 275–280 (1996). [https://doi.org/10.1016/0950-5849\(95\)01059-9](https://doi.org/10.1016/0950-5849(95)01059-9)
23. Cabac, L., Moldt, D.: Formal semantics for AUML agent interaction protocol diagrams. In: *Agent-Oriented Software Engineering V: 5th International Workshop, AOSE 2004*, New York, NY, USA, July 2004, Revised Selected Papers 5, pp. 47–61. Springer (2005). https://doi.org/10.1007/978-3-540-30578-1_4
24. Cabri, G., Leonardi, L., Mariani, S., Zambonelli, F.: Coordination of software agents: models and languages. In: Mascardi and Omicini [67]
25. Caico, R., Cossentino, M., Sabatucci, L., Seidita, V., Gaglio, S.: MetaMeth: a tool for process definition and execution. In: De Paoli et al. [48], pp. 21–24. <http://ceur-ws.org/Vol-204/D06.pdf>
26. Calegari, R., Ciatto, G., Denti, E., Omicini, A., Sartor, G. (eds.): WOA 2020 – 21st Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 2706. Bologna, Italy (2020). <http://ceur-ws.org/Vol-2706/>

27. Calegari, R., Ciatto, G., Denti, E., Omicini, A., Sartor, G. (eds.): WOA 2021 – 22nd Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 2963. Bologna, Italy (2021). <http://ceur-ws.org/Vol-2963/>
28. Chella, A., Cossentino, M., Infantino, I., Pirrone, R.: An agent based design process for cognitive architectures in robotics. In: Omicini and Viroli [74], pp. 84–89. <http://giuseppevizzari.github.io/WOA-proceedings-archive/pdfs/woa2001/pdf/08.pdf>
29. Chella, A., Lanza, F., Seidita, V.: Representing and developing knowledge using Jason, Cartago and OWL. In: Cossentino et al. [42], pp. 147–152. http://ceur-ws.org/Vol-2215/paper_23.pdf
30. Comi, A., Rosaci, D.: SMARTSAN: a P2P social agent network for generating recommendations in a smart city environment. In: De Meo et al. [47], pp. 108–112. <http://ceur-ws.org/Vol-1867/w19.pdf>
31. Corradi, A., Omicini, A., Poggi, A. (eds.): WOA 2000 – 1st Workshop “From Objects to Agents”, Atti di Congressi, vol. 1195. Pitagora Editrice Bologna, Parma, Italy (2000). <http://giuseppevizzari.github.io/WOA-proceedings-archive/woa-2000.html>
32. Cossentino, M., Gaud, N., Hilaire, V., Galland, S., Koukam, A.: ASPECS: an agent-oriented software process for engineering complex systems. *Auton. Agent. Multi-Agent Syst.* **20**(2), 260–300 (2010). <https://doi.org/10.1007/s10458-009-9099-4>
33. Cossentino, M., Hilaire, V., Molesini, A., Seidita, V. (eds.): *Handbook on Agent-Oriented Design Processes*. Springer (2014). <https://doi.org/10.1007/978-3-642-39975-6>
34. Cossentino, M., Hopmans, G., Odell, J.: FIPA standardization activities in the software engineering area. In: Armano et al. [3], pp. 71–77. <http://giuseppevizzari.github.io/WOA-proceedings-archive/pdfs/woa2003/23.pdf>
35. Cossentino, M., Lodato, C., Lopes, S., Ribino, P., Seidita, V., Chella, A.: A UML-based notation for representing MAS organizations. In: Fortino et al. [55], pp. 133–139. http://ceur-ws.org/Vol-741/ID20_CossentinoLodatoLopesRibiniSeiditaChella.pdf
36. Cossentino, M., Lodato, C., Lopes, S., Sabatucci, L.: MUSA: a middleware for user-driven service adaptation. In: Di Napoli et al. [50], pp. 1–10. <http://ceur-ws.org/Vol-1382/paper1.pdf>
37. Cossentino, M., Lopes, S., Sabatucci, L.: A tool for the automatic generation of MOISE organisations from BPMN. In: Calegari et al. [26], pp. 69–82. <http://ceur-ws.org/Vol-2706/paper11.pdf>
38. Cossentino, M., Lopes, S., Sabatucci, L., Tripiciano, M.: Towards a semantic layer for Italian emergency plans. In: Calegari et al. [27], pp. 144–161. <http://ceur-ws.org/Vol-2963/paper9.pdf>
39. Cossentino, M., et al.: GIMT: a tool for ontology and goal modeling in BDI multi-agent design. In: Santoro and Bergenti [85], pp. 81–88. <http://ceur-ws.org/Vol-1260/paper10.pdf>
40. Cossentino, M., Poggi, A., Rimassa, G., Turci, P.: Implementation level issues in MAS modeling. In: Armano et al. [3], pp. 155–162. <http://giuseppevizzari.github.io/WOA-proceedings-archive/pdfs/woa2003/25.pdf>
41. Cossentino, M., Sabatucci, L., Seidita, V.: Towards an approach for engineering complex systems: agents and agility. In: De Meo et al. [47], pp. 1–6. <http://ceur-ws.org/Vol-1867/w1.pdf>
42. Cossentino, M., Sabatucci, L., Seidita, V. (eds.): WOA 2018 – 19th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 2215. Palermo, Italy (2018). <http://ceur-ws.org/Vol-2215/>

43. Cossentino, M., Seidita, V.: PASSI - process for agents society specification and implementation. In: Cossentino et al. [33], pp. 287–329. https://doi.org/10.1007/978-3-642-39975-6_10
44. Dalpiaz, F., Ali, R., Asnar, Y., Bryl, V., Giorgini, P.: Applying Tropos to socio-technical system design and runtime configuration. In: Baldoni et al. [7], pp. 101–107. http://www.pa.icar.cnr.it/woa08/materiali/paper/paper_14.pdf
45. Dalpiaz, F., Molesini, A., Puviani, M., Seidita, V.: Towards filling the gap between AOSE methodologies and infrastructures: requirements and meta-model. In: Baldoni et al. [7], pp. 115–121. http://www.pa.icar.cnr.it/woa08/materiali/paper/paper_3.pdf
46. De Luca, F., Tundis, A., Garro, A.: PROCE: an agent-based PROcess Composition and execution Environment. In: Fortino et al. [55], pp. 171–174. http://ceur-ws.org/Vol-741/DEM02_DeLucaTundisGarro.pdf
47. De Meo, P., Postorino, M.N., Rosaci, D., Sarnè, G.M.L. (eds.): WOA 2017 – 18th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 1867. Scilla, RC, Italy (2017). <http://ceur-ws.org/Vol-1867/>
48. De Paoli, F., Di Stefano, A., Omicini, A., Santoro, C. (eds.): WOA 2006 – 7th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 204. Catania, Italy (2006). <http://ceur-ws.org/Vol-204/>
49. De Paoli, F., Manzoni, S., Poggi, A. (eds.): WOA 2002 – 3rd Workshop “From Objects to Agents”. Pitagora Editrice Bologna, Milano, Italy (2002). <http://giuseppezizzari.github.io/WOA-proceedings-archive/woa-2002.html>
50. Di Napoli, C., Rossi, S., Staffa, M. (eds.): WOA 2015 – 16th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 1382. Naples, Italy (2015). <http://ceur-ws.org/Vol-1382/>
51. Domnori, E., Cabri, G., Leonardi, L.: A multi-agent approach for territorial emergency management. In: Fortino et al. [55], pp. 74–80. http://ceur-ws.org/Vol-741/ID4_DomnoriCabriLeonardi.pdf
52. Drew, D.S.: Multi-agent systems for search and rescue applications. *Curr. Robot. Rep.* **2**, 189–200 (2021). <https://doi.org/10.1007/s43154-021-00048-3>
53. Falcone, R., Castelfranchi, C., Sapienza, A., Cantucci, F. (eds.): WOA 2023 – 24th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 3579. Roma, Italy (2023). <http://ceur-ws.org/Vol-3579/>
54. Ferrando, A., Mascardi, V. (eds.): WOA 2022 – 23rd Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 3261. Genova, Italy (2022). <http://ceur-ws.org/Vol-3261/>
55. Fortino, G., Garro, A., Palopoli, L., Russo, W., Spezzano, G. (eds.): WOA 2011 – 12th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 741. Rende, Italy (2011). <http://ceur-ws.org/Vol-741/>
56. Fortino, G., Garro, A., Russo, W.: Using method engineering for the construction of agent-oriented methodologies. In: Baldoni et al. [8], pp. 51–54. <http://giuseppezizzari.github.io/WOA-proceedings-archive/pdfs/woa2004/8.pdf>
57. Fortino, G., Garro, A., Mascillaro, S., Russo, W.: ELDATool: a statecharts-based tool for prototyping multi-agent systems. In: Baldoni et al. [6], pp. 14–19. http://woa07.dibris.unige.it/papers/D2_ForGarMasRus-WOA07-Demo.pdf
58. Garcia-Ojeda, J.C., DeLoach, S.A., Robby, Oyen, W.H., Valenzuela, J.: O-MaSE: a customizable approach to developing multiagent development processes. In: Agent-Oriented Software Engineering VIII: 8th International Workshop, AOSE 2007, Honolulu, HI, USA, May 2007, Revised Selected Papers 8, pp. 1–15. Springer (2008). https://doi.org/10.1007/978-3-540-79488-2_1

59. Garro, A.: Agent-based computing for science and engineering: back to the future and beyond.... In: Mascardi and Omicini [67]
60. Garro, A., Russo, W.: Exploiting the easyABMS methodology in social and economic domains. In: Bergenti [13], pp. 8–15. <http://www.ailab.unipr.it/woa09/papers/Garro.pdf>
61. Garzetti, M., Giorgini, P., Mylopoulos, J., Sannicolò, F.: Applying Tropos methodology to a real case study: complexity and criticality analysis. In: De Paoli et al. [49], pp. 7–13. <http://giuseppezizzari.github.io/WOA-proceedings-archive/pdfs/woa2002/12.pdf>
62. Hübner, J.F., Sichman, J.S., Boissier, O.: Developing organised multiagent systems using the MOISE⁺ model: programming issues at the system and agent levels. *Int. J. Agent Oriented Softw. Eng.* **1**(3/4), 370–395 (2007). <https://doi.org/10.1504/IJAOSE.2007.016266>
63. Jennings, N.R., Wooldridge, M.J.: *Agent Technology: Foundations, Applications, and Markets*. Springer (1998). <https://doi.org/10.1007/978-3-662-03678-5>
64. Kinny, D., Georgeff, M., Rao, A.: A methodology and modelling technique for systems of BDI agents. In: *Agents Breaking Away: 7th European Workshop on Modelling Autonomous Agents in a Multi-Agent World, MAAMAW '96*, Eindhoven, The Netherlands, January 1996. Proceedings, pp. 56–71. Springer (1996). <https://doi.org/10.1007/BFb0031846>
65. Marguglio, A., Cammarata, G., Bonura, S., Francaviglia, G., Puccio, M., Morreale, V.: Design and development of intentional systems with PRACTIONIST Studio. In: Baldoni et al. [7], pp. 37–45. http://www.pa.icar.cnr.it/woa08/materiali/paper/paper_16.pdf
66. Mariani, S., Omicini, A.: TuCSoN coordination for MAS situatedness: towards a methodology. In: Santoro and Bergenti [85], pp. 48–57. <http://ceur-ws.org/Vol-1260/paper11.pdf>
67. Mascardi, V., Omicini, A. (eds.): *The Agents Journey: Twenty-Five Years of Multi-agent Systems at WOA*. Lecture Notes in Computer Science – State-of-the-Art Surveys. Springer (2026)
68. Molesini, A., Nardini, E., Denti, E., Omicini, A.: Advancing object-oriented standards toward agent-oriented methodologies: SPEM 2.0 on SODA. In: Baldoni et al. [7], pp. 108–114. http://www.pa.icar.cnr.it/woa08/materiali/paper/paper_2.pdf
69. Molesini, A., Omicini, A.: Documenting SODA: an evaluation of the process documentation template. In: Omicini and Viroli [75], pp. 95–101. <http://ceur-ws.org/Vol-621/paper14.pdf>
70. Morreale, V., Bonura, S., Francaviglia, G., Centineo, F., Cossentino, M., Gaglio, S.: Reasoning about goals in BDI agents: the PRACTIONIST framework. In: De Paoli et al. [48], pp. 187–194. <http://ceur-ws.org/Vol-204/P20.pdf>
71. Nardini, E., Viroli, M., Casadei, M., Omicini, A.: A self-organising infrastructure for chemical-semantic coordination: experiments in TuCSoN. In: Omicini and Viroli [75], pp. 117–125. <http://CEUR-WS.org/Vol-621/paper17.pdf>
72. O'Brien, P.D., Nicol, R.C.: FIPA – towards a standard for software agents. *BT Technol. J.* **16**, 51–59 (1998). <https://doi.org/10.1023/A:1009621729979>
73. Omicini, A.: From objects to agent societies: abstractions and methodologies for the engineering of open distributed systems. In: Corradi et al. [31], pp. 29–34. <http://giuseppezizzari.github.io/WOA-proceedings-archive/pdfs/woa2000/WOA07.pdf>
74. Omicini, A., Viroli, M. (eds.): *WOA 2001 – 2nd Workshop “From Objects to Agents”*. Pitagora Editrice Bologna, Modena, Italy (2001). <http://giuseppezizzari.github.io/WOA-proceedings-archive/woa-2001.html>

75. Omicini, A., Viroli, M. (eds.): WOA 2010 – 11th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 621. Rimini, Italy (2010). <http://ceur-ws.org/Vol-621/>
76. Padgham, L., Winikoff, M., DeLoach, S., Cossentino, M.: A unified graphical notation for AOSE. In: Agent-Oriented Software Engineering IX: 9th International Workshop, AOSE 2008, Estoril, Portugal, May 2008, Revised Selected Papers, pp. 116–130. Springer (2008). https://doi.org/10.1007/978-3-642-01338-6_9
77. Pavón, J., Gómez-Sanz, J.J., Fuentes, R.: The INGENIAS methodology and tools. In: Agent-Oriented Methodologies, pp. 236–276. IGI Global (2005). <https://doi.org/10.4018/978-1-59140-581-8.ch009>
78. Perini, A., Bresciani, P., Giorgini, P., Giunchiglia, F., Mylopoulos, J.: Towards an agent oriented approach to software engineering. In: Omicini and Viroli [74], pp. 72–77. <http://giuseppezizzari.github.io/WOA-proceedings-archive/pdfs/woa2001/pdf/19.pdf>
79. Picard, G., Gleizes, M.P.: The ADELFE methodology. In: Methodologies and Software Engineering for Agent Systems: The Agent-oriented Software Engineering Handbook, pp. 157–175. Springer (2004). https://doi.org/10.1007/1-4020-8058-1_11
80. Picasso, E., Postorino, M.N., Sarné, G.M.L.: A study to promote car-sharing by adopting a reputation system in a multi-agent context. In: De Meo et al. [47], pp. 13–18. <http://ceur-ws.org/Vol-1867/w3.pdf>
81. Postorino, M.N., Sarné, G.M.: Agents meet traffic simulation, control and management: a review of selected recent contributions. In: Santoro et al. [86], pp. 112–117. <http://ceur-ws.org/Vol-1664/w19.pdf>
82. Ribino, P., Cossentino, M., Lodato, C., Lopes, S., Sabatucci, L., Seidita, V.: Ontology and goal model in designing BDI multi-agent systems. In: Baldoni et al. [4], pp. 66–72. <http://ceur-ws.org/Vol-1099/paper12.pdf>
83. Russell, S.J., Norvig, P.: Artificial Intelligence: A Modern Approach, Global Edition, 4th Edition. Pearson (2021). <https://elibrary.pearson.de/book/99.150005/9781292401171>
84. Sabatucci, L., Cossentino, M., Gaglio, S.: Building agents with agents and patterns. In: De Paoli et al. [48], pp. 124–129. <http://ceur-ws.org/Vol-204/P19.pdf>
85. Santoro, C., Bergenti, F. (eds.): WOA 2014 – 15th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 1260. Catania, Italy (2014). <http://ceur-ws.org/Vol-1260/>
86. Santoro, C., Messina, F., De Benedetti, M. (eds.): WOA 2016 – 17th Workshop “From Objects to Agents”, CEUR Workshop Proceedings, vol. 1664. Catania, Italy (2016). <http://ceur-ws.org/Vol-1664/>
87. Schaerf, A.: A survey of multi-agent systems for optimization problems. In: Fortino et al. [55], p. 1. http://ceur-ws.org/Vol-741/INV01_Schaerf.pdf
88. Schmidt, D.C.: Model-driven engineering. *Computer* **39**(2), 25–31 (2006). <https://doi.org/10.1109/MC.2006.58>
89. Seidita, V., Cossentino, M., Gaglio, S.: A repository of fragments for agent systems design. In: De Paoli et al. [48], pp. 130–137. <http://ceur-ws.org/Vol-204/P18.pdf>
90. Seidita, V., Sabella, A.M.P., Chella, A.: Agents showing self-disclosure. A preliminary methodological approach. In: Falcone et al. [53], pp. 78–91. <http://ceur-ws.org/Vol-3579/paper6.pdf>
91. Seidita, V., Sabella, A.M.P., Lanza, F., Chella, A.: Agent talks about itself: an implementation using Jason, CArtaGO and Speech Acts. *Intelligenza Artificiale* **17**(1), 7–18 (2023). <https://doi.org/10.3233/IA-230005>

92. Wooldridge, M., Jennings, N.R., Kinny, D.: The Gaia methodology for agent-oriented analysis and design. *Auton. Agent. Multi-Agent Syst.* **3**(3), 285–312 (2000). <https://doi.org/10.1023/A:1010071910869>
93. Zambonelli, F.: Nature-inspired spatial metaphors for pervasive service ecosystems. In: Baldoni et al. [7], pp. 61–67. http://www.pa.icar.cnr.it/woa08/materiali/paper/paper_10.pdf
94. Zambonelli, F.: Towards a discipline of IoT-oriented software engineering. In: Santoro et al. [86], pp. 1–7. <http://ceur-ws.org/Vol-1664/w1.pdf>
95. Zambonelli, F., Viroli, M.: From service-oriented architectures to nature-inspired pervasive service ecosystems. In: Omicini and Viroli [75], pp. 102–109. <http://ceur-ws.org/Vol-621/paper15.pdf>

Open Access This chapter is licensed under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (<http://creativecommons.org/licenses/by-nc-nd/4.0/>), which permits any noncommercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if you modified the licensed material. You do not have permission under this license to share adapted material derived from this chapter or parts of it.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

